

函数高级

引用

```
#include<iostream>
using namespace std;

//引用做函数返回值
int& test1()
{
    int a=10;//局部变量存放在栈区：函数调用完之后释放
    return a;//不要返回局部变量的引用
}

int& test2()
{
    static int a=10;//static:静态变量，存放在全局区，全局区上的数据在程序结束后系统释放
    return a;
}

//引用传递
//void swap3(int &a,int &b)
//{
//    int temp;
//    temp=a;
//    a=b;
//    b=temp;
//}

//地址传递
//void swap2(int *a,int *b)
//{
//    int temp;
//    temp=*a;
//    *a=*b;
//    *b=temp;
//}

//值传递
//void swap1(int a,int b)
//{
//    int temp;
//    temp=a;
//    a=b;
//    b=temp;
//    cout<<a<<endl;
//    cout<<b<<endl;
//}
```

```

void showvalue(const int &val)
{
    //val=1000;加上const后防止误操作修改实参
    cout<<val<<endl;
}

int main()
{
    int a=100;
    showvalue(a);

    // int &ref1=test1();
    // cout<<ref1<<endl; //true
    // cout<<ref1<<endl; //false
    //
    // int &ref2=test2();
    // cout<<ref2<<endl; //true
    // cout<<ref2<<endl; //true
    //
    // test2()=1000;//如果函数的返回值是引用，这个函数调用可以作为左值
    // cout<<ref2<<endl;

    //加上const之后，变为可读但不可修改
    // const int &ref=10;//加上const之后编译器自动进行操作，相当于int temp=10;int
    &ref=temp;
    // ref=20; //false

    // int x=10;
    // int y=20;
    // swap3(x,y);
    // cout<<x<<endl;
    // cout<<y<<endl;
    // return 0;

    // int a=10;
    // int b=20;
    // //int &c;错误，引用必须初始化，而且定义引用是必须等于已经定义的变量，不能等于常数
    // int &c=a;//一旦初始化后，就不可更改
    // cout<<c<<endl;
    // c=b;//将b的值赋给c，不是更改引用
    // cout<<a<<endl;
    // cout<<b<<endl;
    // cout<<c<<endl;

    // int a=10;
    // int &b=a;//引用（取别名）
    // cout<<a<<endl;
    // cout<<b<<endl;
    //
    // b=100;
    // cout<<a<<endl;

```

```
// cout<<b<<endl;  
  
}
```

函数默认参数

```
#include<iostream>  
using namespace std;  
  
int func1(int a,int b,int c)  
{  
    return a+b+c;  
}  
  
int func2(int a,int b=10,int c=20)//如果某个位置已经有了默认参数，那么从这个位置往后从左到  
右都必须有默认值  
{  
    return a+b+c;  
}  
//如果函数声明中有默认参数，函数实现中就不能有默认参数  
//声明和实现中只能有一个默认参数  
  
int main()  
{  
    cout<<func1(10,20,30)<<endl;  
    cout<<func2(10)<<endl;  
    cout<<func2(10,20,30)<<endl;  
    return 0;  
}
```

函数占位参数

```
#include<iostream>  
using namespace std;  
  
void func(int a,int =10)//占位参数  
{//占位参数还可以有默认参数  
    cout<<"this is a function"<<endl;  
}  
  
int main()  
{  
    func(10,10);//后面这个参数必须与相应的形参数据类型相同  
    return 0;  
}
```

函数递归

```
#include<iostream>
using namespace std;

int fac(int n)
{
    int f;
    if(n==0)
    {
        f=1;
    }
    else
    {
        f=fac(n-1)*n;
    }
    return f;
}

int main()
{
    int k;
    cin>>k;
    cout<<fac(k)<<endl;

    return 0;
}
```

函数重载

```
#include<iostream>
using namespace std;

int add(int x,int y)
{
    return x+y;
}

int add(int x,int y,int z)
{
    return x+y+z;
}

double add(double x,double y)
{
    return x+y;
}

double add(double x,double y,double z)
{
    return x+y+z;
}
```

```

}

//引用作为重载的条件
void func(int &a)
{
    cout<<"1"<<endl;
}

void func(const int &a)
{
    cout<<"2"<<endl;
}

//函数重载中默认参数
//void func2(int a,int b=10)
//{
//    cout<<"1"<<endl;
//}
//
//void func2(int a)
//{
//    cout<<"2"<<endl;
//}

int main()
{
    int a=10;

    cout<<add(2,3)<<endl;
    cout<<add(1,2,3)<<endl;
    cout<<add(1.1,2.2)<<endl;
    cout<<add(1.1,2.2,3.3)<<endl;
    func(a); //a可读可更改，所以输出“1”
    func(10); //10为常量，相当于const int &a=10合法，所以输出2；而int &a=10不合法
    //func2(10);出现二义性报错
    return 0;
}

```

内联函数

```

#include<iostream>
using namespace std;

const double pi=3.14;

inline double calarea(double radius)
{
    return pi*radius*radius;
}

int main()

```

```

{
    double r=3.0;
    double area=calarea(r);
    cout<<area<<endl;
    return 0;
}

```

案例一-进制转换

```

#include<iostream>
using namespace std;

double power(int m,int n)
{
    int val=1;
    while(n>0)
    {
        val *= m;
        n--;
    }
    return val;
}

int main()
{
    int value=0;
    cout<<"enter an 8 bit binary number:";

    char ch;
    cin>>ch;

    for(int i=7;i>=0;i--)
    {

        if(ch=='1')
        {
            value += power(2,i);
        }
    }

    cout<<value<<endl;
    return 0;
}

```

案例二-回文数

```
#include<iostream>
using namespace std;

bool sym(int n)
{
    int m=0;
    int i=n;
    while(i>0)
    {
        m=m*10+i%10;
        i=i/10;
    }
    return m==n;
}

int main()
{
    for(int i=11;i<1000;i++)
    {
        if(sym(i)&&sym(i*i)&&sym(i*i*i))
        {
            cout<<i<<" "<<i*i<<" "<<i*i*i<<endl;
        }
    }
    return 0;
}
```